STM – DOCKER E KUBERNETES

Instrutoria Interna

Troubleshooting, Operação Corporativa e Desafio Integrador

Diagnóstico de Falhas, Observabilidade, Boas Práticas e Exercício de Ciclo de Vida Completo

Wescley de Macêdo Sousa
wescleysousa@stm.jus.br

KUBERNETES NA PRÁTICA — OPERAÇÃO, TROUBLESHOOTING E CENÁRIO CORPORATIVO

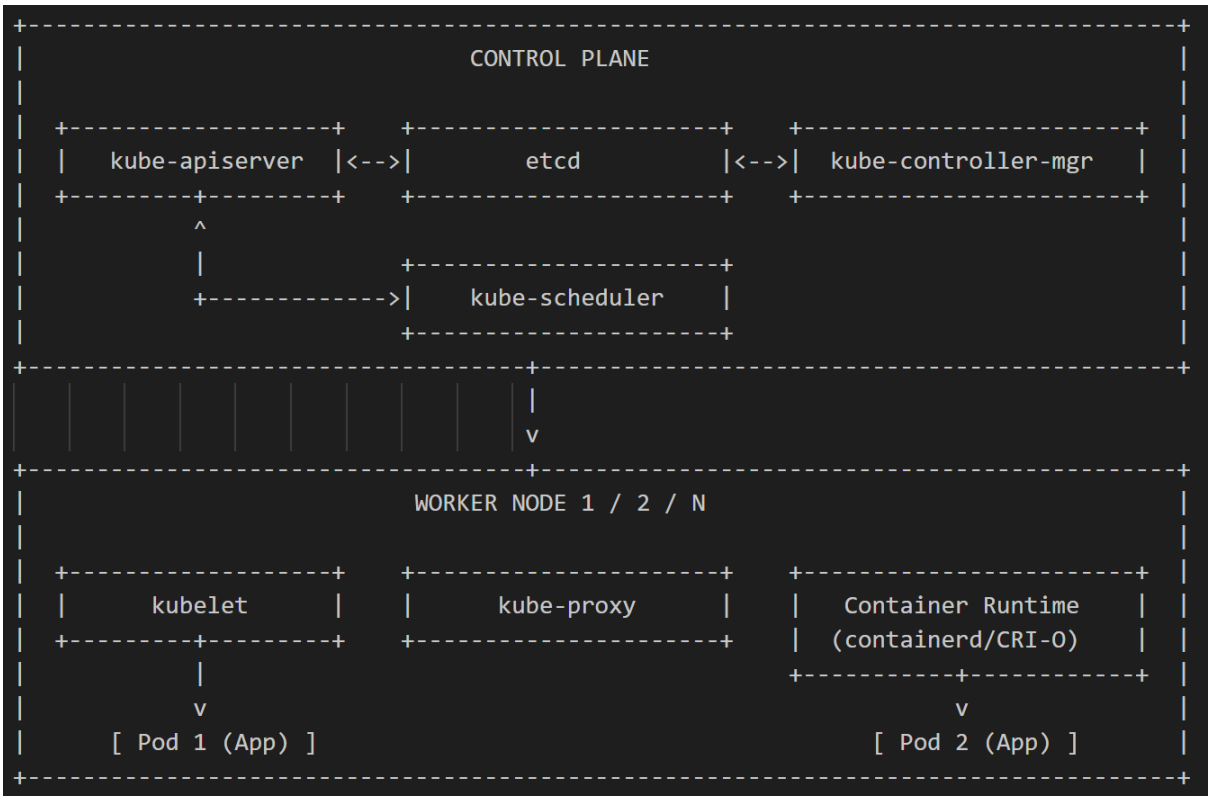
SUMÁRIO

MÓDULO 1: REVISÃO DOS CONCEITOS FUNDAMENTAIS DE ARQUITETURA E CARGAS DE TRABALHO 3
1.1 ARQUITETURA DO KUBERNETES: CONTROL PLANE VS. WORKER NODES 3
1.2 HIERARQUIA DE RECURSOS EM APLICAÇÕES STATELESS 4
1.3 DESCOBERTA DE SERVIÇOS E ABSTRAÇÃO DE REDE 5
1.4 GESTÃO DE CONFIGURAÇÕES E DADOS SENSÍVEIS 5
MÓDULO 2: ENGENHARIA DE TROUBLESHOOTING E DIAGNÓSTICO EM KUBERNETES 6
2.1 METODOLOGIA SISTEMÁTICA DE DIAGNÓSTICO DE FALHAS 6
2.2 ANÁLISE ESTÁTICA E DINÂMICA DE RECURSOS VIA CLI (KUBECTL) 7
2.3 DIAGNÓSTICO AVANÇADO POR ESTADOS CRÍTICOS DO POD 9
2.4 TROUBLESHOOTING DE REDE E COMUNICAÇÃO INTER-SERVIÇOS 11
MÓDULO 3: BOAS PRÁTICAS, SEGURANÇA E OBSERVABILIDADE CORPORATIVA 13
3.1 SEGURANÇA E HARDENING EM PRODUÇÃO 13
3.2 BOAS PRÁTICAS DE CONFIGURAÇÃO E RESILIÊNCIA 14
3.3 OBSERVABILIDADE: PILARES E CONCEITOS INTRODUTÓRIOS 15
3.4 DOCKER + KUBERNETES NO CICLO DE VIDA DAS APLICAÇÕES 16
3.5 VISÃO PRÁTICA E ARQUITETURA NO CONTEXTO CORPORATIVO DO STM 16
MÓDULO 4: ATIVIDADE PRÁTICA INTEGRADORA — PASSO A PASSO GUIADO 18
MÓDULO 5: DESAFIO FINAL CORPORATIVO — GUIA DE RESOLUÇÃO E TROUBLESHOOTING 30
5.1 CENÁRIO CORPORATIVO DO DESAFIO 30
5.2 ROTEIRO DE RESOLUÇÃO DO DESAFIO PRÁTICO 30
5.3 MATRIZ DE COMANDOS DE RESGATE RÁPIDO (CHEATSHEET DO DESAFIO) 32
REFERÊNCIAS 33

MÓDULO 1: REVISÃO DOS CONCEITOS FUNDAMENTAIS DE ARQUITETURA E CARGAS DE TRABALHO

1.1 ARQUITETURA DO KUBERNETES: CONTROL PLANE VS. WORKER NODES

O Kubernetes é uma plataforma de orquestração de containers distribuída e extensível. Sua arquitetura é dividida em duas camadas funcionais distintas: o Control Plane (plano de controle/gestão) e os Worker Nodes (nós de processamento).



Componentes do Control Plane

1. **`kube-apiserver`:** O ponto central de entrada do cluster. Expõe a API RESTful do Kubernetes e valida/configura dados para os objetos (Pods, Services, Deployments). Todos os outros componentes se comunicam exclusivamente através dele.
2. **`etcd`:** Banco de dados chave-valor distribuído e altamente disponível. Armazena todo o estado do cluster, especificações e dados de runtime. É a "fonte da verdade" do Kubernetes.

3. **`kube-scheduler`:** Monitora Pods recém-criados que não possuem um nó atribuído e seleciona o melhor Worker Node para alocá-los, considerando requisitos de recursos, afinidades, *taints* e *tolerations*.
4. **`kube-controller-manager`:** Executa os loops de controle contínuos (ex: *Node Controller*, *ReplicaSet Controller*, *Endpoints Controller*). Garante que o estado real do cluster convirja para o estado desejado.

Componentes dos Worker Nodes

1. **`kubelet`:** Agente principal que roda em cada nó. Recebe as especificações do Pod (`PodSpecs`) via API Server e garante que os containers descritos estejam rodando e saudáveis.
2. **`kube-proxy`:** Proxy de rede que roda em cada nó, mantendo as regras de rede (usando `iptables` ou `IPVS`) para refletir os Services definidos e permitir o roteamento do tráfego para os Pods corretos.
3. **Container Runtime:** O software responsável por baixar as imagens de container e executá-las (ex: `containerd`, `CRI-O`).

1.2 HIERARQUIA DE RECURSOS EM APLICAÇÕES STATELESS

Para aplicações sem estado (*stateless*), o Kubernetes utiliza uma abstração em camadas para garantir alta disponibilidade, facilidade de atualização e tolerância a falhas:

```
[ Deployment ] ──(gerencia/versiona)──► [ ReplicaSet ] ──(garante quantidade)──► [ Pods ]
```

- **Pod:** A menor unidade implantável. Encapsula um ou mais containers que compartilham o mesmo namespace de rede (IP e portas) e volumes de armazenamento.
- **ReplicaSet:** Garante que o número exato de instâncias (*réplicas*) de um Pod idêntico esteja em execução contínua. Caso um Pod falhe, o ReplicaSet agenda uma nova cópia.
- **Deployment:** Fornece declarações de atualização para Pods e ReplicaSets. Permite que você descreva o estado desejado do ciclo de vida da aplicação, gerenciando atualizações progressivas (*Rolling Updates*) e reversões (*Rollbacks*).

1.3 DESCOBERTA DE SERVIÇOS E ABSTRAÇÃO DE REDE

Os Pods são efêmeros e recebem IPs dinâmicos que mudam a cada recriação. O objeto **Service** provê um ponto de acesso fixo e um nome DNS estável para um conjunto de Pods:

Tipo de Service	Escopo	Uso Recomendado
`ClusterIP`	Interno ao Cluster	Comunicação entre microserviços internos (Backend ↔ Banco de Dados).
`NodePort`	Exposição Externa	Expõe uma porta estática (`30000-32767`) no IP de cada Nó Worker. Utilizado em testes ou integração local.
`LoadBalancer`	Exposição Externa	Integra com Load Balancers gerenciados de provedores de nuvem (AWS ALB, GCP LB).
`ExternalName`	Resolução DNS	Redireciona o Service para um CNAME externo (ex: um banco RDS na AWS).

Resolução DNS e CoreDNS

O ecossistema Kubernetes conta com o **CoreDNS** como resolvedor de nomes do cluster. Cada Service ganha um registro DNS no seguinte formato:

$$< nome - servico >.< namespace >.svc.cluster.local$$

1.4 GESTÃO DE CONFIGURAÇÕES E DADOS SENSÍVEIS

Separação estrita entre o código da aplicação e suas configurações (*12-Factor App*):

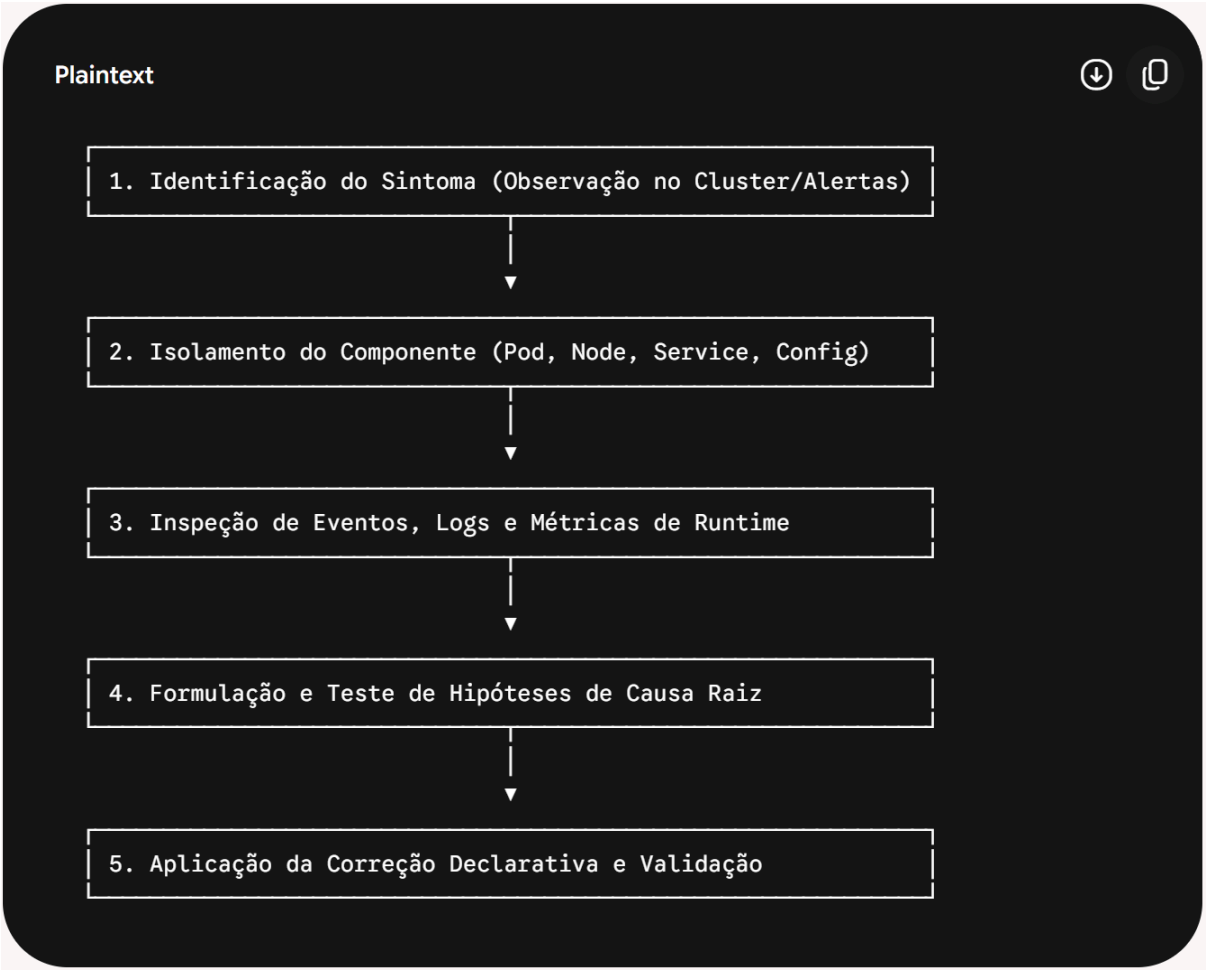
- **ConfigMap:** Armazena dados de configuração não sensíveis em texto claro (arquivos de configuração, variáveis de ambiente, sinalizadores de sistema).

- **Secret:** Armazena dados sensíveis (senhas, chaves privadas, tokens de acesso). No Kubernetes, os dados de um Secret são codificados em **Base64** por padrão no manifesto, devendo ser protegidos via RBAC, criptografia no `etcd` ou integrados a cofres externos como *HashiCorp Vault*.

MÓDULO 2: ENGENHARIA DE TROUBLESHOOTING E DIAGNÓSTICO EM KUBERNETES

2.1 METODOLOGIA SISTEMÁTICA DE DIAGNÓSTICO DE FALHAS

Investigar falhas em um ambiente orquestrado exige uma abordagem metodológica e estruturada. O ciclo de vida do troubleshooting segue cinco etapas em sequência:



2.2 ANÁLISE ESTÁTICA E DINÂMICA DE RECURSOS VIA CLI (KUBECTL)

A ferramenta de linha de comando `kubectl` é o instrumento primário para diagnóstico. Dominar os subcomandos e suas sinalizações é indispensável para respostas rápidas a incidentes.

Comandos Essenciais e Aplicação Prática

1. `kubectl get`

Listagem rápida e estado geral.

```bash
# Listar Pods com detalhes estendidos (IPs, Nós atribuídos, Restarts)
kubectl get pods -n <namespace> -o wide

# Listar todos os recursos de uma só vez para análise geral
kubectl get all -n <namespace>
```

2. `kubectl describe`

Inspeção profunda de metadados, condições, especificações do container e histórico de eventos.

```bash
# Detalhar o estado de um Pod falho
kubectl describe pod <nome-do-pod> -n <namespace>
```

3. `kubectl logs`

Extração de logs de stdout/stderr das aplicações.

```bash
# Logs do container em tempo real
kubectl logs -f <nome-do-pod> -n <namespace>

# Logs do container anterior (útil em Pods reiniciando em CrashLoopBackOff)
kubectl logs <nome-do-pod> --previous -n <namespace>

# Logs de um container específico em Pods multi-containers
kubectl logs <nome-do-pod> -c <nome-do-container> -n <namespace>
```

4. `kubectl exec`

Depuração dinâmica dentro do container em execução.

```bash
# Abrir um terminal interativo dentro do Pod
kubectl exec -it <nome-do-pod> -n <namespace> -- /bin/sh
```

5. `kubectl top`

Monitoramento do consumo de recursos computacionais em tempo real (exige o *Metrics Server* instalado no cluster).

```bash
# Verificar consumo de CPU e Memória por Pod
kubectl top pods -n <namespace>

# Verificar consumo por Nó Worker
kubectl top nodes
```

2.3 DIAGNÓSTICO AVANÇADO POR ESTADOS CRÍTICOS DO POD

Abaixo estão detalhadas as falhas mais frequentes encontradas na operação do Kubernetes, suas causas raízes e estratégias diretas de resolução.

DIAGNÓSTICO POR ESTADOS DO POD

Status do Pod	Causa Raiz Provável	Comando Chave
ImagePullBackOff	Tag/Imagem errada ou Secret	kubectl describe
CrashLoopBackOff	Erro na aplicação/Config ausente	kubectl logs
OOMKilled	Limite de memória estourado	kubectl describe
Pending	Recursos insuficientes/Taints	kubectl describe
CreateContainerConfigError	ConfigMap/Secret não encontrado	kubectl describe

1. `ImagePullBackOff` / `ErrImagePull`

- **Sintoma:** O Pod permanece travado tentando baixar a imagem sem sucesso.
- **Causas Frequentes:**
  1. O nome ou a tag da imagem Docker no arquivo YAML está incorreto ou não existe no registry.
  2. O repositório de imagens é privado e o Pod não possui o `imagePullSecrets` configurado.
  3. Falha de conectividade de rede entre o Worker Node e o Registry de Imagens.
- **Fluxo de Investigação:**

```bash
kubectl describe pod <nome-do-pod>
# Olhar a seção 'Events' no final do output
```

- **Resolução:** Corrigir a imagem no manifesto YAML ou criar o Secret com credenciais de autenticação (`docker-registry`).

2. `CrashLoopBackOff`

- **Sintoma:** O container é iniciado, falha, encerra sua execução, e o Kubernetes tenta reiniciá-lo continuamente com intervalos cada vez maiores (*backoff delay*).
- **Causas Frequentes:**
  1. A aplicação lança uma exceção não tratada ao iniciar (ex: erro de sintaxe, conexão recusada com banco de dados).
  2. A variável de ambiente obrigatória não foi injetada.
  3. O comando de inicialização (`ENTRYPOINT` ou `CMD`) termina imediatamente.
- **Fluxo de Investigação:**

```bash
# Inspecione os logs do processo que falhou logo antes de cair
kubectl logs <nome-do-pod> --previous
```

3. `OOMKilled` (Out of Memory Killed - Exit Code 137)

- **Sintoma:** O container é sumariamente exterminado pelo Kernel do Linux (*Out of Memory Killer*).
- **Causas Frequentes:**
  1. O consumo de memória RAM do container ultrapassou o limite estipulado em `resources.limits.memory`.
  2. Vazamento de memória (*memory leak*) no código da aplicação.
- **Fluxo de Investigação:**

```
kubectl describe pod <nome-do-pod>
# Procure na seção 'Last State' por: Terminated - Reason: OOMKilled - Exit Code: 137
```

- **Resolução:** Ajustar os limites de memória no manifesto ou otimizar a aplicação.

4. `Pending`

- **Sintoma:** O Pod foi aceito pela API, mas não consegue ser agendado (*scheduled*) em nenhum nó.

- **Causas Frequentes:**
    1. **Recursos Insuficientes:** A soma das requisições de CPU/Memória (`requests`) dos Pods ultrapassa a capacidade livre dos nós.
    2. **Taints e Tolerations:** O nó possui uma rejeição (*Taint*) e o Pod não declara a tolerância correspondente.
    3. **NodeSelector / Affinity:** Nenhum nó corresponde às labels exigidas pelo Pod.
    4. **Volume Binding:** O Pod está aguardando um volume persistente (PV/PVC) ser provisionado.
- **Fluxo de Investigação:**

```bash
kubectl describe pod <nome-do-pod>
# Na seção 'Events', verifique mensagens emitidas pelo 'default-scheduler'
```

5. `CreateContainerConfigError` / `CreateContainerError`

- **Sintoma:** O Kubernetes não consegue montar o container devido a erros de configuração de dependências.
- **Causas Frequentes:**
    1. O Pod faz referência a um `ConfigMap` ou `Secret` que ainda não foi criado no cluster.
    2. Uma chave declarada no `valueFrom` não existe dentro do `ConfigMap` ou `Secret` referenciado.

2.4 TROUBLESHOOTING DE REDE E COMUNICAÇÃO INTER-SERVIÇOS

Falhas de conectividade entre componentes são causas frequentes de indisponibilidade.

Checklist de Verificação de Conectividade

1. **Validar Seletores do Service:** O selector do Service deve corresponder exatamente às labels do Pod.

```bash
# Verificar se existem IPs atribuídos no objeto Endpoints
kubectl get endpoints <nome-do-servico>
```

Se o campo ENDPOINTS retornar <none>, o Service não está encontrando os Pods!
Validação de Portas:

- `port`: Porta exposta pelo Service (usada pelos clientes internos).
- `targetPort`: Porta em que a aplicação realmente escuta dentro do container.

Diagnóstico via Pod Temporário de Depuração: Para testar a rede a partir de dentro do cluster, suba uma imagem utilitária de rede:

```bash
kubectl run nettool --rm -i --tty --image=nicolaka/netshoot -- /bin/bash

# Dentro do container do nettool:
# Testar resolução DNS do Service
nslookup <nome-do-servico>.<namespace>.svc.cluster.local

# Testar requisição HTTP
curl -v http://<nome-do-servico>:<porta>
```

MÓDULO 3: BOAS PRÁTICAS, SEGURANÇA E OBSERVABILIDADE CORPORATIVA

3.1 SEGURANÇA E HARDENING EM PRODUÇÃO

Garantir a segurança em cluster Kubernetes envolve a aplicação do princípio do menor privilégio (*Least Privilege*) em múltiplas camadas.

1. Role-Based Access Control (RBAC)

Mecanismo para controlar quem pode realizar operações na API do Kubernetes.

- **`Role` / `ClusterRole`:** Define um conjunto de permissões (verbos: `get`, `list`, `watch`, `create`, `delete`) sobre recursos específicos (Pods, Services, Secrets).
- **`RoleBinding` / `ClusterRoleBinding`:** Associa a Role a um usuário, grupo ou `ServiceAccount`.

2. Configuração de Contexto de Segurança (`SecurityContext`)

Aplica configurações de isolamento diretamente no nível do Pod ou do Container:

```yaml
spec:
  securityContext:
    runAsNonRoot: true              # Impede rigorosamente a execução como usuário root
    runAsUser: 10001                # Força o ID do usuário Linux não privilegiado
    runAsGroup: 10001
    fsGroup: 10001
  containers:
    - name: app
      image: minha-imagem:1.0.0
      securityContext:
        allowPrivilegeEscalation: false # Impede escalada de privilégios via SUID
        readOnlyRootFilesystem: true     # Torna o sistema de arquivos do container somente leitura
        capabilities:
          drop:
            - ALL                       # Remove todas as capacidades do Kernel Linux
```

3. NetworkPolicies

Por padrão, a rede do Kubernetes é plana e sem restrições: **qualquer Pod pode se comunicar com qualquer outro Pod no cluster**. As `NetworkPolicies` atuam como um Firewall para isolar a comunicação de rede (Ingress/Egress) por namespace ou labels.

3.2 BOAS PRÁTICAS DE CONFIGURAÇÃO E RESILIÊNCIA

1. Definição Estrita de Recursos Computacionais

A ausência de limites de recursos permite que uma aplicação ruidosa tombe o nó (*noisy neighbor problem*).

2. Sondas de Saúde (*Health Probes*)

Permitem que o `kubelet` monitore a saúde da aplicação e tome ações automáticas:

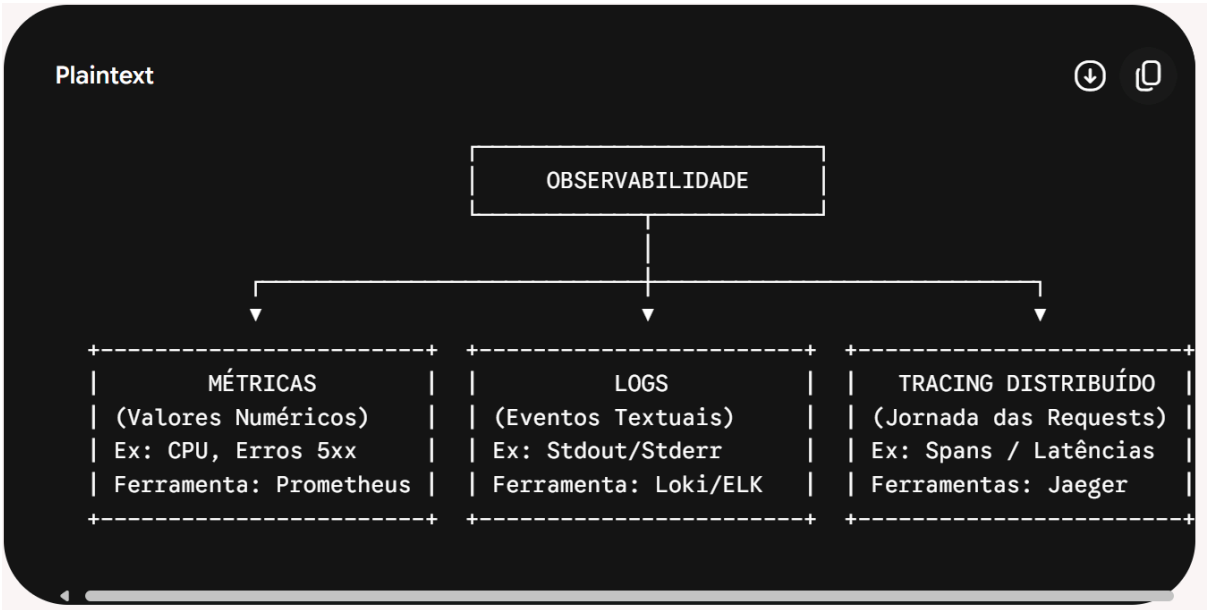
- **`livenessProbe`:** Verifica se a aplicação está viva. Se falhar, o `kubelet` mata e reinicia o container.
- **`readinessProbe`:** Verifica se a aplicação está pronta para receber tráfego. Se falhar, o Service remove o IP do Pod de seus *Endpoints*, parando o envio de requisições.
- **`startupProbe`:** Desativa temporariamente as verificações de liveness/readiness enquanto aplicações legadas realizam sua inicialização inicial lenta.

YAML

```yaml
readinessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 20
```

3.3 OBSERVABILIDADE: PILARES E CONCEITOS INTRODUTÓRIOS

A observabilidade em sistemas distribuídos refere-se à capacidade de inferir o estado interno de um sistema analisando suas saídas externas. Ela se apoia em **Três Pilares**:



1. **Métricas (O que está acontecendo?):** Dados numéricos agregados ao longo do tempo (ex: taxas de erro, requisições por segundo, uso de memória). A ferramenta padrão da indústria para coleta de métricas em Kubernetes é o **Prometheus**, com visualização no **Grafana**.
2. **Logs (Por que está acontecendo?):** Registros contendo contexto de um evento pontual. Em ecossistemas Kubernetes, utiliza-se a arquitetura de *Centralized Logging* (ex: **Grafana Loki** ou **Elasticsearch/Fluentd/Kibana - EFK**).
3. **Tracing Distribuído (Onde está o gargalo?):** Rastreia o caminho de uma requisição que atravessa múltiplos microserviços. Ferramentas comuns incluem **Jaeger** e **OpenTelemetry**.

3.4 DOCKER + KUBERNETES NO CICLO DE VIDA DAS APLICAÇÕES

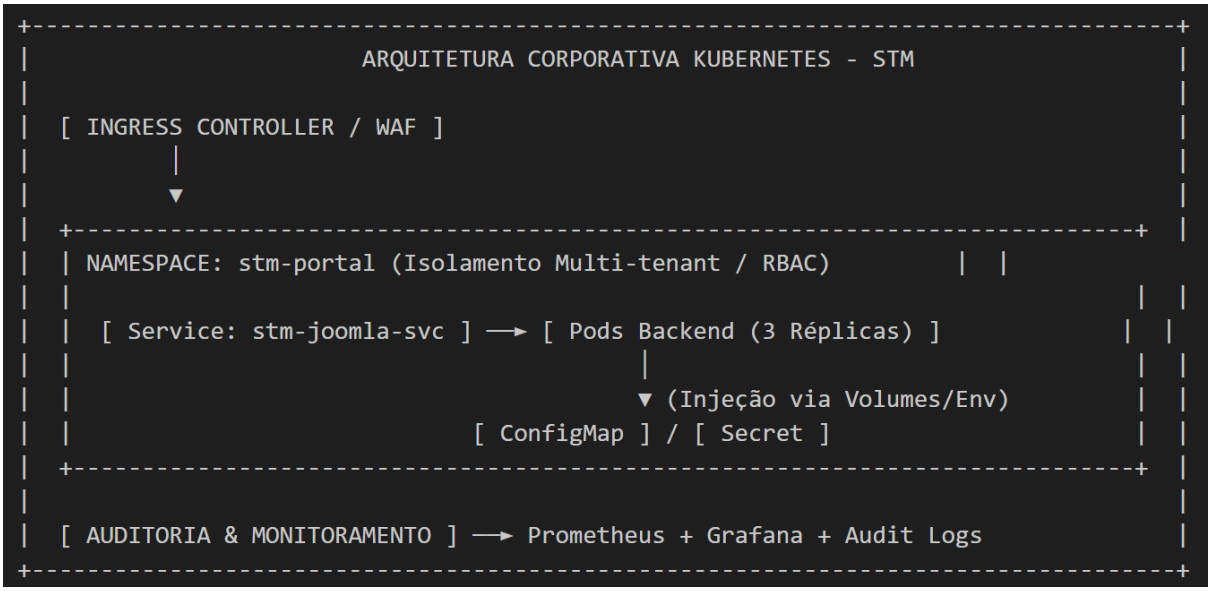
O ciclo de vida moderno de engenharia de software integra containers desde o desenvolvimento local até a produção via pipelines de CI/CD:



1. **Desenvolvimento Local:** O desenvolvedor codifica a aplicação e cria o `Dockerfile`.
2. **Integração Contínua (CI):** A cada `push` no repositório, o pipeline compila o código, executa testes unitários, realiza o *build* da imagem Docker e envia o artefato imutável para um Registry Seguro com uma tag única (ex: `v1.2.0` ou `commit-sha`).
3. **Entrega Contínua (CD/GitOps):** Ferramentas como **ArgoCD** ou **Flux** detectam a atualização dos manifestos no repositório Git e sincronizam o estado no cluster Kubernetes de forma automatizada.

3.5 VISÃO PRÁTICA E ARQUITETURA NO CONTEXTO CORPORATIVO DO STM

O Superior Tribunal Militar (STM) opera em um ambiente judiciário crítico, onde a integridade, a auditabilidade e a disponibilidade contínua dos sistemas de informação são requisitos legais mandatários.



Requisitos e Padrões Arquiteturais no STM:

- **Isolamento via Namespaces:** Separação estrita de sistemas estratégicos (ex: *Processo Judicial Eletrônico*, *Sistemas Administrativos*) em namespaces distintos.
- **Conformidade de Segurança:** Imagens de container assinadas digitalmente, escaneamento contínuo de vulnerabilidades (*Trivy/Clair*) e proibição absoluta de execução de containers como `root`.
- **Alta Disponibilidade e Resiliência:** Ambientes distribuídos geograficamente com estratégias de *RollingUpdate* e políticas de *Autoscaling* (HPA) para absorver picos de acesso.
- **Auditabilidade Operacional:** Registros detalhados de logs e eventos de API mantidos para auditoria conforme as diretrizes do Poder Judiciário.

MÓDULO 4: ATIVIDADE PRÁTICA INTEGRADORA — PASSO A PASSO GUIADO

Nesta atividade integradora, você irá construir do zero uma aplicação web corporativa, empacotá-la em uma imagem Docker, criar seus manifestos no Kubernetes, realizar o deploy, aplicar uma atualização, diagnosticar uma falha proposital de produção e executar a reversão (*rollback*).

Passo 1: Criação da Aplicação Web e `Dockerfile`

Crie um diretório de trabalho chamado `laboratorio-integrador`:

```bash
mkdir laboratorio-integrador && cd laboratorio-integrador
```

1. Código da Aplicação (app.py)

Crie o arquivo **app.py**:

```
import os
import sys
from flask import Flask

app = Flask(__name__)

# Carregamento das variáveis de ambiente
ENV = os.getenv("APP_ENV", "desconhecido")
WELCOME_MSG = os.getenv("WELCOME_MSG", "Mensagem padrao")
DB_USER = os.getenv("DB_USER", "nao_definido")

# Falha proposital para simulação de bug em runtime
SIMULATE_CRASH = os.getenv("SIMULATE_CRASH", "false")

if SIMULATE_CRASH.lower() == "true":
    print("ERRO CRÍTICO: Falha simulada de inicialização na aplicação!", file=sys.stderr)
    sys.exit(1)

@app.route('/')
def home():
    return f"""
    <h1>Sistema Corporativo - STM</h1>
    <p><b>Ambiente:</b> {ENV}</p>
    <p><b>Mensagem:</b> {WELCOME_MSG}</p>
    <p><b>Usuario BD:</b> {DB_USER}</p>
    """

@app.route('/healthz')
def health():
    return "OK", 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

2. Arquivo de Dependências (`requirements.txt`)

Crie o arquivo **`requirements.txt`**:

Plaintext

```
Flask==3.0.0
```

3. O Dockerfile (`Dockerfile`)

Crie o arquivo **`Dockerfile`**:

Dockerfile

```
FROM python:3.11-slim

# Definir diretório de trabalho
WORKDIR /app

# Copiar dependências e instalar
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar o código da aplicação
COPY app.py .

# Criar usuário não-root para segurança
RUN useradd -u 10001 appuser
USER appuser

# Expor a porta da aplicação
EXPOSE 5000

# Comando de inicialização
CMD ["python", "app.py"]
```

Passo 2: Build da Imagem Docker e Teste Local

Execute o build da imagem informando a tag `stm-app:v1.0.0`:

```bash
docker build -t stm-app:v1.0.0 .
```

Testar a Imagem Localmente:

```bash
docker run -d --name teste-stm -p 5000:5000 \
  -e APP_ENV="Local-Docker" \
  -e WELCOME_MSG="Teste local realizado com sucesso" \
  stm-app:v1.0.0
```

Valide no seu navegador ou terminal acessando `http://localhost:5000`. Em seguida, pare e remova o container local:

```bash
docker stop teste-stm && docker rm teste-stm
```

*(Nota: Se você estiver utilizando o **Kind** como seu cluster local, carregue a imagem para dentro do cluster executando: `kind load docker-image stm-app:v1.0.0`)*

Passo 3: Elaboração dos Manifestos Kubernetes

1. Manifestos de Configuração (`01-configmap.yaml` e `02-secret.yaml`)

Crie o arquivo **`01-configmap.yaml`**:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: stm-app-config
  namespace: default
data:
  APP_ENV: "Producao-STM"
  WELCOME_MSG: "Bem-vindo ao Portal de Serviços da Justiça Militar"
  SIMULATE_CRASH: "false"
```

Crie o arquivo `02-secret.yaml`:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: stm-app-secret
  namespace: default
type: Opaque
data:
  # DB_USER = "admin_stm" codificado em Base64
  DB_USER: YWRtaW5fc3Rt
```

2. Manifesto do Deployment (`03-deployment.yaml`)

Crie o arquivo `03-deployment.yaml`:

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stm-app-deployment
  namespace: default
  labels:
    app: stm-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: stm-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: stm-app
    spec:
      containers:
        - name: stm-container
          image: stm-app:v1.0.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: stm-app-config
            - secretRef:
                name: stm-app-secret
```

```
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "250m"
        memory: "256Mi"
    readinessProbe:
      httpGet:
        path: /healthz
        port: 5000
      initialDelaySeconds: 5
      periodSeconds: 5
    livenessProbe:
      httpGet:
        path: /healthz
        port: 5000
      initialDelaySeconds: 10
      periodSeconds: 10
```

3. Manifesto do Service (`04-service.yaml`)

Crie o arquivo `04-service.yaml`:

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: stm-app-service
  namespace: default
spec:
  type: NodePort
  selector:
    app: stm-app
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 5000
      nodePort: 30080
```

Passo 4: Implantação e Operação de Produção

1. Aplicar os Manifestos no Cluster:

Bash

```bash
kubectl apply -f 01-configmap.yaml
kubectl apply -f 02-secret.yaml
kubectl apply -f 03-deployment.yaml
kubectl apply -f 04-service.yaml
```

2. Validar a Implantação:

```bash
kubectl get pods -l app=stm-app
kubectl get service stm-app-service
```

3. Testar a Escalabilidade:

Aumente o número de réplicas da aplicação de 2 para 4:

```bash
kubectl scale deployment stm-app-deployment --replicas=4
```

Verifique o aumento das réplicas ativas executando `kubectl get pods`.

Passo 5: Simulação de Falha, Diagnóstico e Rollback

Agora simularemos uma falha crítica de atualização em ambiente corporativo.

1. Induzir a Falha de Configuração

Altere o arquivo `01-configmap.yaml`, modificando a chave `SIMULATE_CRASH` para `"true"`:

```yaml
# Trecho alterado no 01-configmap.yaml
data:
  SIMULATE_CRASH: "true"
```

Aplique a alteração no ConfigMap e force a reinicialização dos Pods para carregar a nova configuração:

```bash
kubectl apply -f 01-configmap.yaml
kubectl rollout restart deployment stm-app-deployment
```

2. Diagnóstico da Falha via CLI

Observe que os novos Pods entram no estado de **`CrashLoopBackOff`** ou **`Error`**:

```bash
kubectl get pods
```

Executar a investigação sistemática aprendida no Módulo 2:

- **Passo A: Inspeção do Evento e Condição do Pod**

```bash
kubectl describe pod -l app=stm-app
```

Resultado: O evento relata que o container falhou continuamente e a sonda de `readiness` falhou.

Passo B: Inspeção dos Logs do Container

```bash
kubectl logs -l app=stm-app --tail=20
```

Saída dos Logs:

```
ERRO CRÍTICO: Falha simulada de inicialização na aplicação!
```

3. Execução do Rollback

Identificada a falha crítica, acione a reversão do Deployment para a versão funcional anterior:

```bash
kubectl rollout undo deployment stm-app-deployment
```

Também reverta a variável do ConfigMap para `"false"` e reaplique-a:

```bash
# Corrija o 01-configmap.yaml para SIMULATE_CRASH: "false" e aplique:
kubectl apply -f 01-configmap.yaml
```

4. Validação da Restauração

```bash
kubectl rollout status deployment stm-app-deployment
kubectl get pods
```

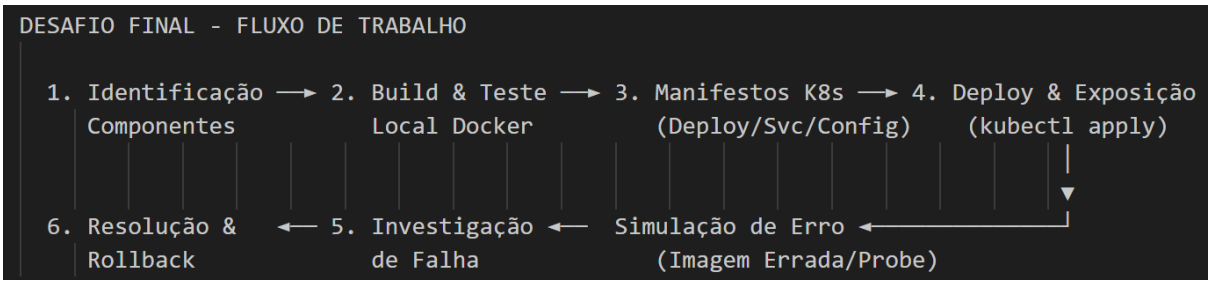
Saída: Todos os Pods retornam ao estado `Running` e `1/1 READY`.

MÓDULO 5: DESAFIO FINAL CORPORATIVO — GUIA DE RESOLUÇÃO E TROUBLESHOOTING

5.1 CENÁRIO CORPORATIVO DO DESAFIO

Você assumiu a sustentação da infraestrutura de aplicações da Justiça Militar do STM. O time de desenvolvimento disponibilizou um novo microserviço corporativo em container.

Sua missão é atuar como Engenheiro de Plataforma/DevOps para colocar a aplicação em produção, garantir a resiliência e resolver um problema simulado que impedirá a execução correta da aplicação.



5.2 ROTEIRO DE RESOLUÇÃO DO DESAFIO PRÁTICO

Siga as etapas abaixo para completar com sucesso o Desafio Final:

Etapa 1: Reconhecimento e Análise da Aplicação

1. Inspecione a estrutura de arquivos da aplicação recebida.
2. Identifique quais variáveis de ambiente a aplicação exige para conectar (ex: portas, credenciais, hosts).
3. Verifique se existe um `Dockerfile` e quais portas ele expõe.

Etapa 2: Empacotamento Docker e Execução Local

1. Gere a imagem Docker localmente com uma tag de versão: `docker build -t stm-desafio:v1.0.0 .`
2. Teste a imagem localmente passando as variáveis de ambiente necessárias via `docker run`.
3. Certifique-se de que o *endpoint* de saúde `/healthz` ou `/` responde corretamente.

Etapa 3: Elaboração dos Manifestos Kubernetes

Construa um conjunto unificado de arquivos YAML contemplando:

1. **ConfigMap:** Contendo todas as configurações da aplicação.
2. **Secret:** Contendo senhas/tokens sensíveis em Base64.
3. **Deployment:**
   - Mínimo de 3 réplicas.
   - Recursos declarados (`requests` e `limits`).
   - Injeção de variáveis via `envFrom`.
   - Probes de `livenessProbe` e `readinessProbe` configuradas na porta correta da aplicação.
4. **Service:** Do tipo `NodePort` mapeando a porta do serviço para os Pods.

Etapa 4: Implantação e Escalonamento

1. Aplique todos os manifestos no cluster: `kubectl apply -f .`
2. Verifique se os Pods estão no estado `Running` e `1/1 READY`.
3. Escale o deployment para 5 réplicas: `kubectl scale deployment <nome-deployment> --replicas=5`.

Etapa 5: Troubleshooting de Incidente Simulado

Durante o desafio, uma atualização com falha será simulada executando o comando:

```bash
kubectl set image deployment/<nome-deployment> <nome-container>=stm-desafio:v2.0.0-inexistente
```

Seu Roteiro de Ação para o Diagnóstico:

1. Identifique o estado dos Pods travados com `kubectl get pods`.
2. Diagnostique a causa exata com `kubectl describe pod <pod-falho>`.
3. Confirme o erro de pull de imagem (`ImagePullBackOff`).
4. Execute a reversão do ambiente imediata com `kubectl rollout undo deployment/<nome-deployment>`.
5. Valide no terminal que o tráfego voltou a ser servido perfeitamente nas 5 réplicas funcionais.

5.3 MATRIZ DE COMANDOS DE RESGATE RÁPIDO (CHEATSHEET DO DESAFIO)

Guarde esta matriz de comandos para utilização ágil durante o desafio e na operação do dia a dia:

Bash

```
# === INSPEÇÃO E ESTADO GENERALIZADO ===
kubectl get pods -o wide --show-labels
kubectl get events --sort-by='.metadata.creationTimestamp' -n default

# === DETALHAMENTO E DIAGNÓSTICO PROFUNDO ===
kubectl describe pod <nome-do-pod>
kubectl describe node <nome-do-no>

# === EXTRAÇÃO DE LOGS DE RUNTIME ===
kubectl logs <nome-do-pod> --all-containers=true
kubectl logs <nome-do-pod> -p   # Logs do container que crashou anteriormente

# === INTERAÇÃO E DEPURAÇÃO DE REDE ===
kubectl exec -it <nome-do-pod> -- /bin/sh
kubectl get ep <nome-do-servico>

# === CICLO DE VIDA E ROLLOUT ===
kubectl rollout status deployment/<nome-deployment>
kubectl rollout history deployment/<nome-deployment>
kubectl rollout undo deployment/<nome-deployment>
```

REFERÊNCIAS

Livros e Obras de Referência

BURNS, Brendan; BEDA, Joe; HIGHTOWER, Kelsey; EVENSON, Lachlan. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. 3. ed. Sebastopol: O'Reilly Media, 2022.

IBRYAM, Bilgin; HUß, Roland. *Kubernetes Patterns: Reusable Elements for Designing Cloud-Native Applications*. 2. ed. Sebastopol: O'Reilly Media, 2023.

HIGHTOWER, Kelsey; BURNS, Brendan; BEDA, Joe. *Kubernetes: Up and Running*. 2. ed. Sebastopol: O'Reilly Media, 2019.

LUKSA, Marko. *Kubernetes in Action*. 2. ed. Manning Publications, 2023.

MAYOR, Michael; HERNANDEZ, Loris. *Kubernetes Security and Observability*. Packt Publishing, 2022.

PULTORAK, David. *Docker and Kubernetes for Java Developers*. Packt Publishing, 2021.

SAYFAN, Gigi. *Mastering Kubernetes*. 4. ed. Packt Publishing, 2023.

Recursos Web e Documentação Oficial

CNCF — CLOUD NATIVE COMPUTING FOUNDATION. *Cloud Native Computing Foundation*. Disponível em: https://www.cncf.io/ . Acesso em: 11 out. 2026.

HELM. *Helm Documentation: The Package Manager for Kubernetes*. Disponível em: https://helm.sh/docs/ . Acesso em: 11 out. 2026.

KUBERNETES. *Kubernetes Documentation*. Disponível em: https://kubernetes.io/docs/ . Acesso em: 11 out. 2026.

KUBERNETES. *Concepts: Cluster Architecture, Workloads, Networking, Storage and Security*. Disponível em: https://kubernetes.io/docs/concepts/ . Acesso em: 11 out. 2026.

KUBERNETES. *Tasks: Tutorials and Administrative Procedures*. Disponível em: https://kubernetes.io/docs/tasks/ . Acesso em: 11 out. 2026.

KUBERNETES. *kubectl Reference Documentation*. Disponível em: https://kubernetes.io/docs/reference/kubectl/ . Acesso em: 11 out. 2026.

MINIKUBE. *Minikube Documentation*. Disponível em: https://minikube.sigs.k8s.io/docs/ . Acesso em: 11 out. 2026.

PROMETHEUS. *Prometheus Documentation*. Disponível em: https://prometheus.io/docs/ . Acesso em: 11 out. 2026.

Referências Complementares para DevOps e Cloud Native

ARGO PROJECT. *Argo CD Documentation*. Disponível em: https://argo-cd.readthedocs.io/ . Acesso em: 11 out. 2026.

CNCF. *Cloud Native Landscape*. Disponível em: https://landscape.cncf.io/ . Acesso em: 11 out. 2026.

DOCKER. *Docker Documentation*. Disponível em: https://docs.docker.com/ . Acesso em: 11 out. 2026.

OPEN CONTAINER INITIATIVE (OCI). *Image Specification and Runtime Specification*. Disponível em: https://opencontainers.org/ . Acesso em: 11 out. 2026.

SIGSTORE. *Sigstore Documentation*. Disponível em: https://docs.sigstore.dev/ . Acesso em: 11 out. 2026.